

第5章 中間コード生成

5.1 概要

5.1.1 逆ポーランド記法

逆ポーランド記法は引数の数が固定の演算を後置形式で記載する。計算に用いる値をメモリの値を `push` 命令で参照することによって計算することができ、算術式の間コードとして古くから用いられている。逆ポーランド記法からより低レベルの機械語（バイナリコード）へ変換することは容易である。演算の引数となる変数を *ID* で参照するとき、`push ID` でその値をスタックに積む、演算を適用するとき、*n* 引数の演算では、*n* 個の値をスタックから `pop` して演算を適用し、その結果をスタックに `push` する。例えば、 $x+y*z$ という入力に対しては、`push x, push y, push z, *, +` という命令列によってスタックの先頭に値の計算が積まれる (図 5.1)。

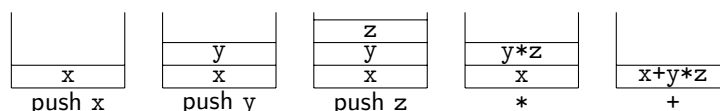


図 5.1: 逆ポーランド記法の計算

実際には構文木の構成において、 x, y, z はそれぞれトークンとして *ID* として扱われるが、実際のコード生成においては、変数名を必要とする。字句解析においてもトークンの他に実際の識別子の記号列が保存されている。(lex における `yytext` など)

5.1.2 抽象構文木

抽象構文木 (abstract syntax tree) は、構文解析木をコード生成に必要な部分のみを残して変形した木構造である。コード生成において、構文解析木に現れる非終端記号の情報は必要でない場合が多く、生成規則の持つ意味をノードに記述した方がコード生成には都合が良い。木構造がコードの意味を直接的に表現していると考えられることから、抽象構文木が中間コードであるとみなすことができる。例えば、代入式 $w=x+y*z$ の抽象構文木は図 5.2 のようになる。

5.1.3 3 番地コード

3 番地コード (3-address code) は、命令を一般的に $C = A \theta B$ の形に限定して動作を記述する。ここで、 A, B, C はメモリ番地であり、 A のメモリの内容と B のメモリの内容に θ の演算を適用し

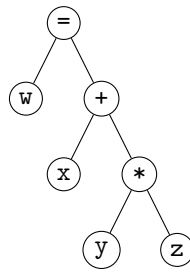


図 5.2: 抽象構文木の例

て、結果を C の番地に格納することを表す。これ以外に単項演算、条件付きジャンプ命令などを用いることによって一般的な計算を単純な命令の組み合わせで記述することができる。各命令にはラベル L をつけることができ、ジャンプ命令の飛び先とすることができる。3 番地コードを機械語に変換することは容易であり、3 番地コードによって最適化を行うことでハードウェアに依存しないコード最適化を適用することができる。

```

int fib(int x) {
    if (x=0) {
        return 0;
    } else if (x=1) {
        return 1;
    } else {
        return fib(x-2)+fib(x-1);
    }
}

print fib(4);

```

```

define @fib(x) {
    t1 := x
    t2 := (x=0)
    if not t1 goto l1
    return 0
l1: t3 := (x=1)
    if not t2 goto l2
    return 1
l2: t4 := t1 - 2
    t5 := call fib(t4)
    t6 := t1 - 1
    t7 := call fib(t6)
    t8 := t5 + t7
    return t8
}

define @main() {
    t1 := 4
    t2 := call fib(t1)
    print t2
}

```

図 5.3: 3 番地コードへの変換例

図 5.3 に変換例を示す。ここでは、関数呼び出しとして@関数名 という形をとっている。

LLVM-IR

LLVM はコンパイラ研究でよく用いられる研究基盤である。3 番地コードを基本として中間コードとして LLVM-IR が提案されている。LLVM-IR では静的単一代入形式でデータフローが式の解析によって得られ、最適化を容易にしている。図 5.4 に変換例を示す。

```
int fib(int x) {  
    if (x=0) {  
        return 0;  
    } else if (x=1) {  
        return 1;  
    } else {  
        return fib(x-2)+fib(x-1);  
    }  
}  
  
print fib(4);
```

```
; ModuleID = 'main.c'  
source_filename = "main.c"  
  
@.str = private unnamed_addr @backslash\  
        constant [4 x i8] c"%d\0A\00", align 1  
  
; Function Attrs: noinline nounwind optnone uwtable  
define i32 @fib(i32) #0 {  
    %2 = alloca i32, align 4  
    store i32 %0, i32* %2, align 4  
    %3 = load i32, i32* %2, align 4  
    %4 = icmp eq i32 %3, 0  
    br i1 %4, label %5, label %6  
  
5:  
    ret i32 0  
  
6:  
    %7 = icmp eq i32 %3, 1  
    br i1 %7, label %8, label %9  
  
8:  
    ret i32 1  
  
9:  
    %10 = load i32, i32* %2, align 4  
    %11 = sub nsw i32 %10, 2  
    %12 = call i32 @fib(i32 %11)  
    %13 = load i32, i32* %2, align 4  
    %14 = sub nsw i32 %13, 1  
    %15 = call i32 @fib(i32 %14)  
    %16 = add i32 %12, %15  
    ret i32 %16  
}
```

図 5.4: LLVM-IR への変換例

$%n$ は、レジスタ n を表す。中間コードでは、レジスタは任意数あるとする。LLVM-IR は、代入ごとにレジスタの番号を変えることによって静的単一代入 (static single assignment, SSA) を実現する。LLVM-IR は実用的な中間コードであり、基本的なデータ型やビット長などが詳細に記述される。以下に主な命令を示す。(「きつねさんにもわかる LLVM」より引用)

alloca	result=alloca(type) [,type) (NumElements)] [,align(alignment)]	メモリ確保命令 type × NumElements の領域を関数のスタックフレーム上に確保
load	result=load [volatile] (type)* (pointer) [,align(alignment)] [,!nontemporal!(index)] [,!invariant.load !(index)] result=load atomic [volatile] (type)* (pointer) (singlethread) (ordering) !index=! i32 1 align (alignment)	ロード命令 pointer が指すメモリアドレスから値を読み出す volatile が指定されている場合、最適化は load 命令の実行順や実行数をほかの volatile な命令に変更することができない
store	store [volatile] (type) (value) (type)* (pointer) [,align (alignment)] [,!nontemporal !(index)] store atomic [volatile] (type) (value) (type)* (pointer) [singlethread] (ordering) align (alignment)	ストア命令 pointer が指すメモリアドレスに値を格納する volatile が指定されている場合、最適化は store 命令の実行順や実行数をほかの volatile な命令に変更することができない
getelementptr	result = getelementptr (pttype)* (ptrval) {, (type) (idx) }* result = getelementptr inbounds (pttype)* (ptrval) {, (type) (idx) }* result = getelementptr (ptr vector), (vector index type) (idx)	要素アドレス取得命令 合成型内の要素のアドレスを計算する ptrval が指すアドレスをベースとして idx で指定したインデックスの要素アドレスを計算する

図 5.5: LLVMIR(メモリアクセス)

add	result = add (type) (op1) (op2) result = add nuw (type) (op1) (op2) result = add nsw (type) (op1) (op2) result = add nuw nsw (type) (op1) (op2)	加算命令 op1 と op2 を加算して結果を返す。op1, op2 は整数か整数のベクタ。 nuw = No Unsigned Wrap, nsw = No Signed Wrap/ オーバーフローが発生したときに結果が未定義
------------	--	---

同様な演算命令 fadd, sub, fsub, mul, fmul, udiv, sdiv, fdiv, urem,srem,frem(剰余)

図 5.6: LLVMIR(演算命令)

ret	ret (type) (value) ret void	return 命令 関数の呼び出し元へ制御を戻す value が指定されていれば value を戻す
br	br il (cond), label (iftrue), label (iffalse) br label (dest)	分岐命令 cond の true/false に応じて分岐先が変わる cond が指定されない場合はラベル dest に無条件分岐する
icmp	result=icmp (cond) (type) (op1) (op2)	比較命令 op1, op2 を cond で指定した条件で比較し、真偽を返す。 op1, op2 は整数、ポインタか整数のベクトル
call	result = [tail] call [cconv] [ret attrs] (type) [(< fnty)*] (fnptrval) ((function args))[fn attrs]	関数呼び出し命令 関数 fnptrval を function args を引数として呼び出す。

図 5.7: LLVMIR(制御命令)

eq	equal 2つの引数が同値の場合 true, それ以外 false
ne	equal の否定
ugt	2つの引数を unsigned value として扱って第一引数が第二引数よりも大きい場合に true, それ以外 false
uge	2つの引数を unsigned value として扱って第一引数が第二引数よりも大きいか同じ場合に true, それ以外 false
ult	2つの引数を unsigned value として扱って第一引数が第二引数よりも小さい場合に true, それ以外 false
ule	2つの引数を unsigned value として扱って第一引数が第二引数よりも小さいか同じ場合に true, それ以外 false
sgt	2つの引数を unsigned value として扱って第一引数が第二引数よりも大きい場合に true, それ以外 false
sge	2つの引数を unsigned value として扱って第一引数が第二引数よりも大きいか同じ場合に true, それ以外 false
slt	2つの引数を unsigned value として扱って第一引数が第二引数よりも小さい場合に true, それ以外 false
sle	2つの引数を unsigned value として扱って第一引数が第二引数よりも小さいか同じ場合に true, それ以外 false

図 5.8: LLVMIR(比較命令)

5.2 実行環境

プログラミング言語として局所変数を持つことが一般的である。オブジェクト指向言語では、インスタンスが生成されると変数領域を割り当てて実行が終了したとき領域を解放する必要がある。命令型では、関数・手続きが呼び出されたときには、関数・手続きのみで有効な局所変数を割り当てることでプログラムを合成的に記述することが可能になっている。局所変数は関数・手続きの呼び出しごとに動的に割り当てられることで、再帰的な関数定義が可能になっている。関数と手続きの呼び出しは、入れ子構造であることから、**駆動レコード** (Activation record) という単位でスタック構造を操作することで実行環境を実現する。単純に**スタックフレーム** (stack frame) と呼ばれることもある。

5.2.1 駆動レコードの構成

プログラムを実行するプロセスの全体のメモリは図 5.9a のような形となる場合が多い。駆動レコードの構成は一般には図 5.9b のような構成となる。

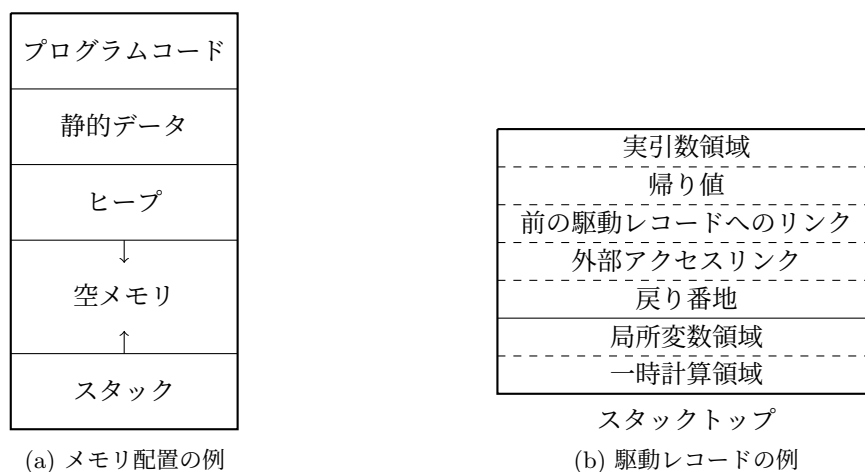


図 5.9: 実行環境のメモリ配置

1. 実引数領域：引数を評価して値を格納して関数・手続きに引き渡す。
2. 返り値：関数の計算が終了して得られる値を格納する。
3. 駆動リンクへのリンク：スタックの格納単位を示す。
4. 外部アクセスリンク：大域変数などへの参照のための番地を格納する。
5. 戻り番地：関数・手続きが終了したときに実行するプログラムカウンタの番地を格納する。
6. 局所変数領域：関数・手続きの実行時に割り当てられる変数領域を示す。
7. 一時計算領域：関数・手続きの実行に必要な一時変数領域を示す。

駆動レコードを単位として、プロセスのメモリ領域に割り当てられる。空きメモリの量はプロセス起動時に決定されることが多い。十分な量を見込んで割り当てられるが、無限ループなどで異常に多くの関数呼び出しが発生するとプロセスのメモリ限界を超える。この場合は、通常、オペレーティングシステムが異常を検出してプロセスが異常終了する。

関数呼び出しとリターン

関数（手続き）の呼び出しは以下の順序で行われる。

1. スタック上に駆動レコードを確保する。
2. 実引数の値を評価して、呼び出しが起こる駆動レコードの領域に書き込む。
3. 呼び出しが発生したプログラムカウンタ PC の値+1 をスタックに格納
4. 前の駆動レコードの先頭のリンクを格納
5. 局所領域をスタック上に確保
6. 呼び出された関数の命令の頭にジャンプする。

関数（手続き）において、リターンが実行されたとき。

1. 関数であれば返し値を駆動レコードの領域に格納
2. 戻りアドレスを駆動レコードから取り出す。
3. 駆動レコードの先頭のリンクを取り出して、現在の駆動レコードを破棄
4. 戻りアドレスへジャンプ

例 5.1. 以下の C 言語風のプログラムに対する実行環境の概要は図??のようになる。ここでは、駆動レコードのうちの変数領域のみの状況を示している。Rv は関数の返り値を表す。

```

int a;

int func(int x) {
    int y;
    if (x == 1) { return(a); }
    else {
        y = func(x-1) * 2;
    }
    return (a+y); }

void main() {
    int b;
    b = 3;
    a = 2;
    printf("%d\n",func(b)); }

```

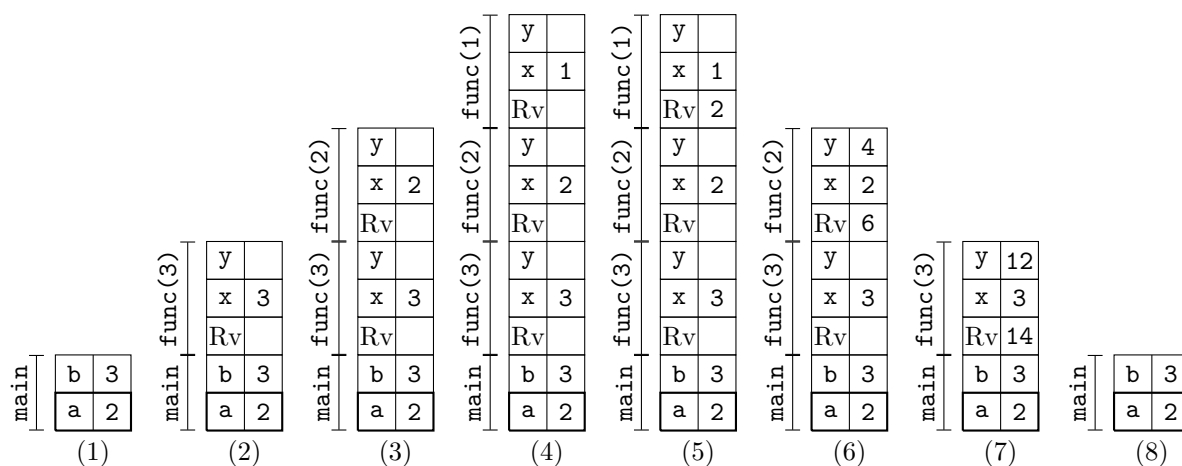


図 5.10: 実行環境の変化

(1) `main` の実行中(2) `func(3)` の呼び出し直後(3) `func(3)` から `func(2)` を呼び出した直後(4) `func(2)` から `func(1)` を呼び出した直後(5) `func(1)` の戻り値を `a` の値として 2 をセット(6) `func(1)` の駆動レコードを破棄して, `func(2)` の戻り値を $2 * 2 + 2$ で 6 にセット(7) `func(2)` の駆動レコードを破棄して, `func(3)` の戻り値を $6 * 2 + 2$ で 14 にセット

(8) func(3) の駆動レコードを破棄.

5.3 構文主導型変換

中間コードは構文解析の進行とともに生成される. このような方法を**構文主導型変換** (syntax-directed translation, SDT) と呼ぶ. この変換では生成規則に対してコード生成ルーチンを割り当てて, 規則の適用にしたがって対応するコード生成ルーチンを呼び出して中間コードを生成する.

5.3.1 SDT の基礎

コード生成ルーチンとして以下の3つの関数を用いる.

- **newTemp(& T)**
 T にあたらな領域を割り当ててそのメモリ番地を T に束縛する. **newTemp(& x)** を実行すると, x に新たに割り当てられた領域をさすメモリ番地が割り当てられる. 順番に割り当てられる領域を $t1, t2, t3, \dots$ で参照する.
- **newLabel(& L)**
 新たなラベルをわりあててそのラベルの命令番地を L に格納する. 割り当てられる領域を $L1, L2, \dots$ で参照する.
- **generate**
 S 引数の3番地コードを生成して出力に書き込む. 各コードとの対応を以下のように定める.

$C = A\theta B$	generate ($C, =, A, \theta, B$)
$B = \theta A$	generate ($B, =, \theta, A$)
$B = A$	generate ($B, =, A$)
if A goto L	generate (if. A, L)
if_not A goto L	generate (if_not. A, L)
goto L	generate (goto. $L, $)
$Li :$	generate (L)

例 5.2. 以下の生成規則を持つ CF 文法を考える.

$$\begin{aligned} S &\rightarrow ID = E ; \\ E &\rightarrow E^{(1)} + NUM \\ E &\rightarrow NUM \end{aligned}$$

中間コード生成は, 生成規則に生成ルーチンを対応づけることで実現される.

生成規則	コード生成ルーチン
$E \rightarrow \text{NUM}$	$\{ \text{newTemp}(\&E.var);$ $\text{generate}(E.var, =, \text{NUM.val}); \}$
$E \rightarrow E^{(1)} + \text{NUM}$	$\{ \text{newTemp}(\&E.var)$ $\text{generate}(E.var, =, E^{(1)}.var, +, \text{NUM.val}); \}$
$S \rightarrow \text{ID} = E ;$	$\{ \text{generate}(\text{ID.name}, =, E.var) \}$

ここで、 $E.var$ は、非終端記号 E に対して割り当てられる変数、 NUM.val はトークン NUM が表す数値、 ID.name は、識別子 ID の名前を表す文字列を表す。

$\text{ID} = \text{NUM}_0 + \text{NUM}_1 + \text{NUM}_2 ;$ に対する中間コード生成は以下の通りである。

還元系列	生成中間コード
$\text{ID} = \text{NUM}_0 + \text{NUM}_1 + \text{NUM}_2 ;$	$t1 = \text{NUM}_0.val \ (E_1.var = t1)$
$\Leftarrow \text{ID} = E_1 + \text{NUM}_1 + \text{NUM}_2 ;$	$t2 = t1 + \text{NUM}_1.val \ (E_2.var = t2)$
$\Leftarrow \text{ID} = E_2 + \text{NUM}_2 ;$	$t3 = t2 + \text{NUM}_2.val \ (E_3.var = t3)$
$\Leftarrow \text{ID} = E_3 ;$	$\text{ID.name} = t3$
$\Leftarrow S$	

実行制御を含む文に対する SDT

条件文のコード生成は以下のようになる。

$$S \rightarrow \text{if} (E) S^{(1)} \Rightarrow \begin{array}{l} E \text{ のコードを生成 } E.var = t1 \\ \text{if not } t1 \text{ goto } L1 \\ S^{(1)} \text{ のコードを生成} \\ L1 \end{array}$$

構文解析を生成規則に沿って進めるためには、条件文の生成規則が適用されたときに全体のコードを生成するのではなく、条件式 E が還元されたときに、 E のコードを生成して出力する必要がある。

$$\begin{array}{l} S \rightarrow \text{if} (E) \{ \text{newLabel}(\&S.label); \\ \quad \text{generate}(\text{if-not}.E.var, S.label); \} \\ S^{(1)} \quad \{ \text{generate}(S.label :); \} \end{array}$$

While 文に対する構文主導型変換は以下の順番に行われる。図??に変換つきのルールを示す。

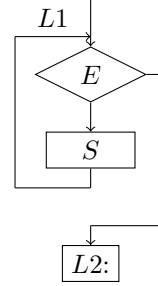
$$S \rightarrow \text{while} (E) S^{(1)}$$

この生成規則に対する中間コードは以下のようになる。

```

1: L1:
2: E のコード (t1=E.var)
3: if-not t1 goto L2
4: S(1) のコード
5: goto L1
6: L2:

```



コード生成ルーチンを伴う生成規則は以下のようになる。

```

S  →  while ( {newLabel(&S.label1);
               generate(S.label1)}
            E ) {newLabel(&S.label2)
               generate(if-not.&E.var, goto S.label2)
S(1) {generate(goto S.label1);
      generate(S.label2)}

```

5.3.2 上昇型構文主導型変換

LR パーザーにコード生成ルーチンを組み込む場合、コード生成は還元のタイミングで行う。したがって、条件分岐、ループのように生成規則の途中でコード生成を行う必要がある場合には、新たな非終端記号を導入すれば還元の際にのみコード生成ルーチンを呼び出すことができる。

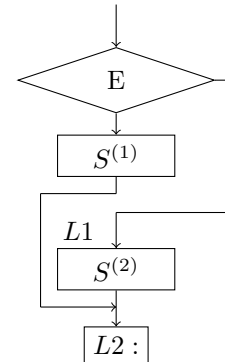
$$S \rightarrow \text{if } (E) S^1 \text{ else } S^{(2)}$$

についてはフローチャートに沿って以下のような生成ルーチンを付加する。

```

S  →  if ( E ) {newLabel(S.label1);
               generate(if-not.E.var, goto S.label1)}
S(1) {newLabel(&S.label2);
      generate(goto S.label2);
      {generate(S.label1)}
else S(2) {generate(S.label2)}

```



$$S \rightarrow IS', \quad I \rightarrow \text{if } (E), \quad S' \rightarrow S^1 S'', \quad S'' \rightarrow \text{else } S^{(2)}$$

$$\begin{aligned}
 I &\rightarrow \text{if} (E) && \{ \text{newLabel}(S.\text{label1}); \\
 &&& \text{generate}(\text{if-not}.E.\text{var}, \text{goto } S.\text{label1}) \} \\
 S' &\rightarrow IS^{(1)} && \{ \text{newLabel}(\&S'.\text{label}); \\
 &&& \text{generate}(\text{goto } S'.\text{label}); \\
 &&& \text{generate}(I.\text{label}); \\
 S &\rightarrow S' \text{ else } S^{(2)} && \{ \text{generate}(S'.\text{label}) \}
 \end{aligned}$$

while 文については以下のような生成規則が構成される.

$$\begin{aligned}
 I &\rightarrow \text{while} (&& \{ \text{newLabel}(\&I.\text{label}); \\
 &&& \text{generate}(I.\text{label}) \} \\
 S' &\rightarrow E) && \{ \text{newLabel}(\&S'.\text{label}); \\
 &&& \text{generate}(\text{if-not}.E.\text{var}, \text{goto } S'.\text{label}) \} \\
 S &\rightarrow IS'S^{(1)} && \{ \text{generate}(\text{goto } I.\text{label}); \\
 &&& \text{generate}(S'.\text{label})
 \end{aligned}$$

5.3.3 下降型構文主導型変換

下降型構文解析にコード生成に組み込む場合は、より直観的にコード生成することができる. 生成規則の右辺で生成ルーチンに到達すれば、そのルーチンを実行すればよい.

5.3.4 属性文法